Credit Card Fraud Detection

using Machine Learning

Machine Learning · End-Semester Project · B.Tech Computer Science and Engineering

Akshita Modi · AP24110010386
Lakshay · AP24110010677
Mohana Krishna Reddy Pittu · AP24110011262
Nadimpally Dheva Harshit · AP24110011553

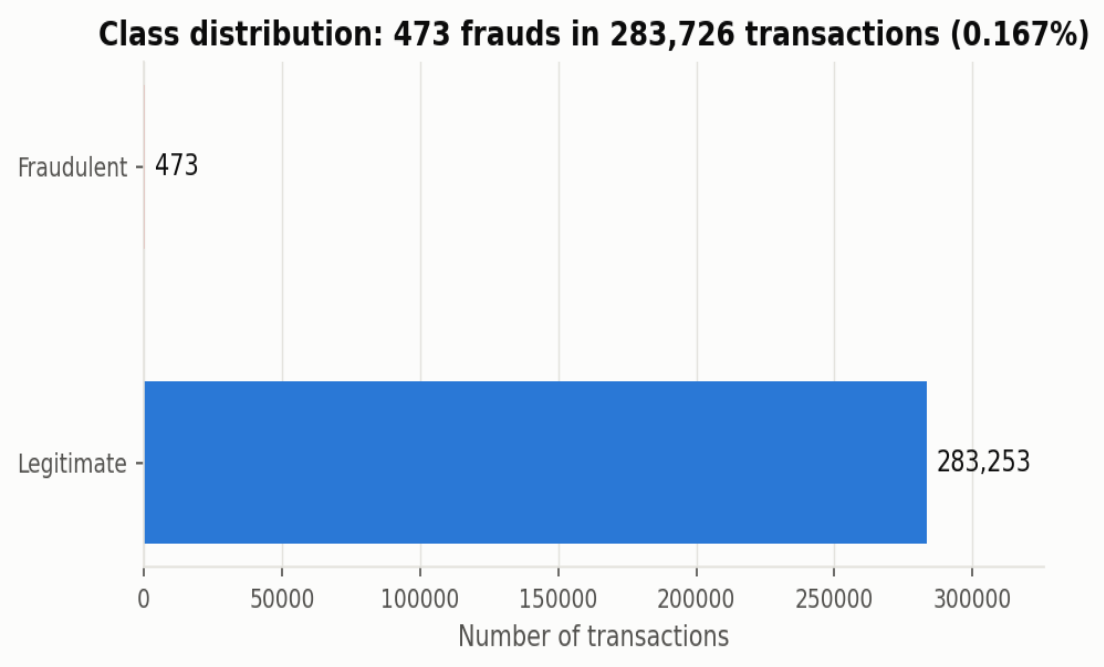
Supervisor
Mr. Ajay Dilip Kumar Marapatla
SRM University – AP

1 What this project does

Problem

and why it is harder than it looks

- A bank processes millions of card transactions a day. A tiny fraction are fraudulent, and checking each by hand is impossible.
- The task is binary classification: given a transaction's attributes, is it fraud?
- The difficulty is not the classification. It is the extreme class imbalance — fraud is 0.17% of the data, about 1 in every 600.
- The two mistakes cost very different amounts:
  - A missed fraud (false negative) costs the bank the full transaction value.
  - A false alarm (false positive) costs one analyst one review.
- Objective: catch as much fraud as possible while keeping false alarms low enough for a real team to handle.



The problem, in one number

99.83%

is the accuracy of a classifier that predicts “legitimate” for every single transaction.

It catches zero fraud.

Accuracy therefore appears nowhere in our results. Every model is judged on recall, precision, F1-score and the area under the precision–recall curve.

2 The dataset

Data

ULB Credit Card Fraud Detection benchmark (Kaggle)

- Real card transactions by European cardholders over two days, September 2013.
- Published by the Machine Learning Group, Université Libre de Bruxelles.
- 31 columns, no missing values:
  - V1–V28 — principal components. The bank could not release raw features for privacy, so PCA was applied before publication.
  - Time, Amount — the only untransformed attributes. Class — the target.
- We removed 1,081 exact duplicate rows first — a duplicate in both train and test would leak.

284,807	1,081	283,726	473
raw	duplicates	used	frauds (0.1667%)

Split: stratified 70 / 30, random seed 77

- Training: 198,608 transactions, 331 frauds
- Test: 85,118 transactions, 142 frauds
- Seed 77 = last two digits of roll number AP24110010677, as instructed.
- The same split is reused by every model, so the comparison is fair.

3 How we ran it, and how we measured it

Method

One protocol for every model · four metrics, none of them accuracy

The protocol

1. Deduplicate, then split — no row in both halves
2. One shared stratified 70/30 split, seed 77, imported by every notebook
3. Scaling inside a Pipeline — fitted on the training fold only
4. Resampling inside the CV folds, never before the split
5. Five-fold stratified cross validation to confirm the ranking
6. Decision threshold derived from cost, not left at 0.5

The metrics

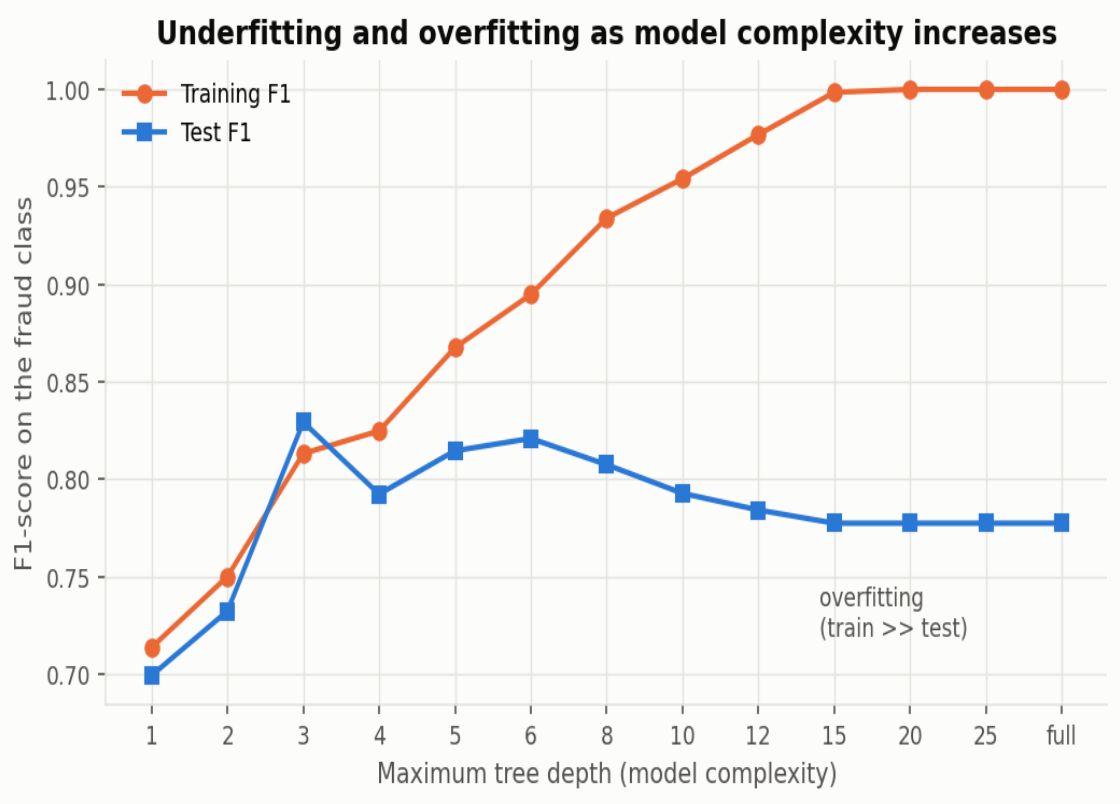
Recall	Of all frauds, how many did we catch?
Precision	Of all alerts, how many were real?
F1-score	Harmonic mean — one number for ranking
PR-AUC	The right summary at very low prevalence
Confusion matrix	Cost in transactions, not percentages

Data leakage is the most common flaw in published fraud work — steps 1, 3 and 4 exist to prevent it. ROC-AUC is avoided because its denominator is the enormous legitimate class, which flatters weak models at 0.17% prevalence.

Unit I — Foundations

Training, testing, validation, overfitting

- Supervised vs unsupervised learning: eleven supervised models, plus K-means as an unsupervised check.
- Train / test / validation separation, written once in Code/utils.py and shared by every notebook.
- Five-fold stratified cross validation — stratified because an ordinary fold could contain almost no frauds.
- Overfitting and underfitting demonstrated on the data rather than described:
  - A shallow tree underfits. A deep unpruned tree memorises the training set and generalises worse.
  - The widening gap between the training and test curve is what pruning depth was chosen to close.



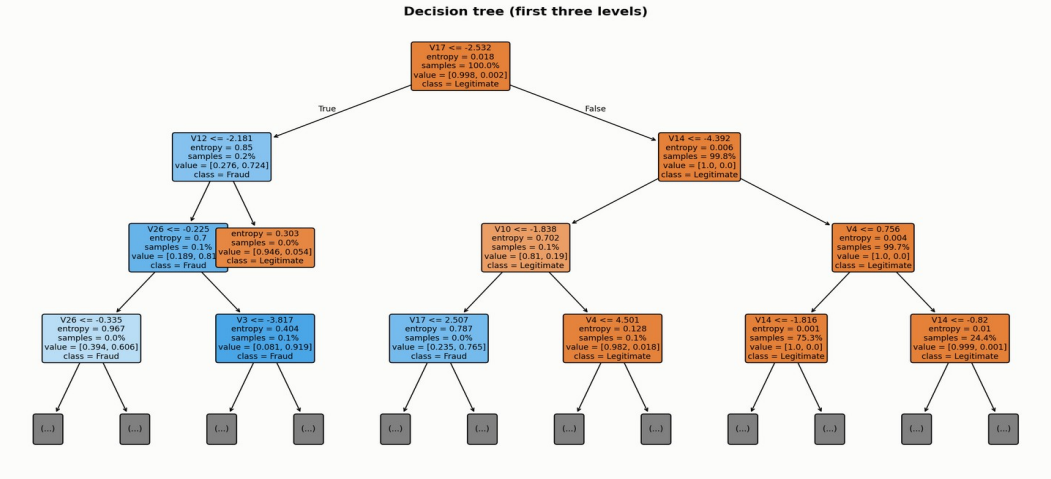
5

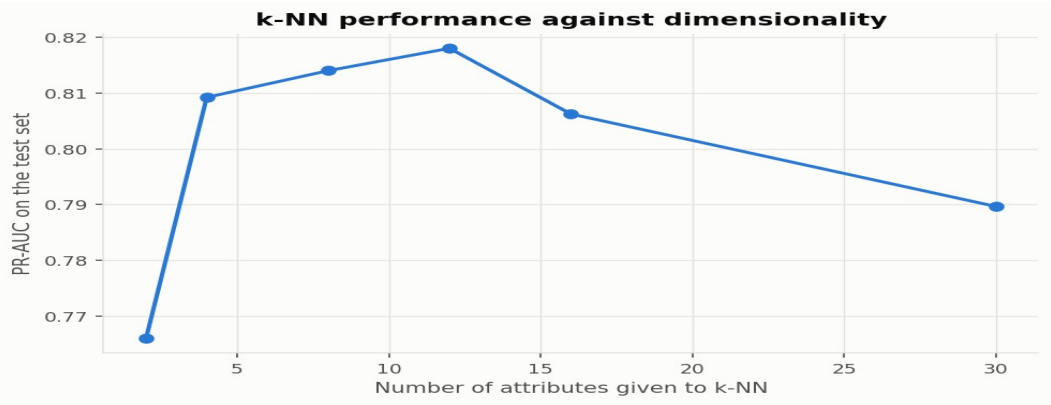
Unit II — Trees, instance-based learning, feature reduction

Unit II

Decision tree (ID3) · k-NN · selection · PCA and LDA

- Decision tree — ID3: entropy and information gain, computed by hand for the first split and checked against scikit-learn. Depth chosen by CV (depth 6, 29 leaves, root split V17 ≤ −2.53).
  - The only model whose decision can be read directly as a rule. F1 0.822, 111 of 142 caught.
- k-Nearest Neighbour — a lazy learner with no training phase; it classifies by distance at prediction time. F1 0.800.
  - Used to demonstrate the curse of dimensionality on real data: as attributes are added, distances converge and performance degrades.
- Feature selection: SelectKBest (F-test), mutual information, Recursive Feature Elimination.
- Dimensionality reduction: PCA (unsupervised, maximum variance) and LDA (supervised, maximum class separation).
  - A reduced feature set retained most of the detection quality, but no reduction beat using all 30 features.





6

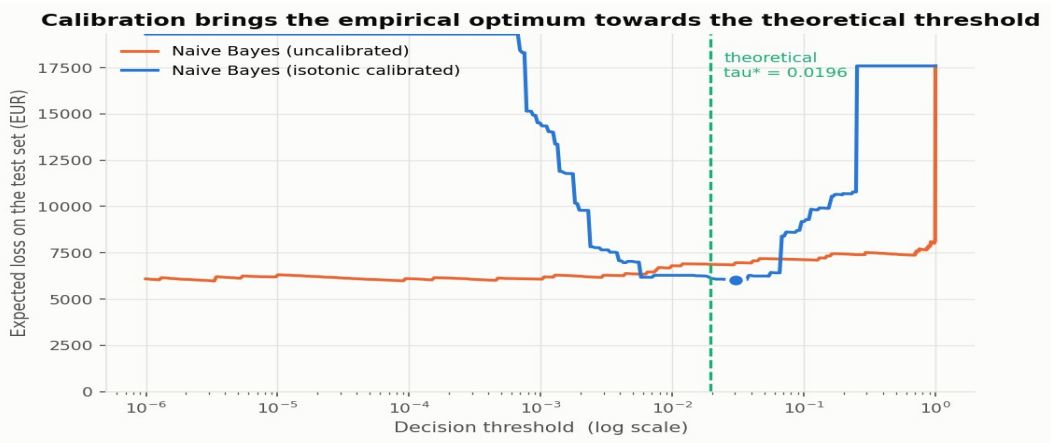
Unit III — Probabilistic and margin-based learning

Unit III

Naive Bayes · Bayes optimal decisions · calibration · SVM

- Gaussian naive Bayes on Bayes theorem. We tested the independence assumption rather than asserting it — the features are not independent, and the model suffers: recall 0.775 but precision 0.113, with 865 false alarms.
- Bayes optimal decisions give the threshold directly from cost. If a false negative costs 50× a false positive, the break-even point is $C^{fp}/(C^{fp}+C^{fn}) = 1/51 \approx 0.02$ — flag anything above 2%, not 50%.
- It did not work at first. The derivation assumes genuine probabilities, and naive Bayes saturates towards 0 and 1.
  - Isotonic calibration fixed it, and the empirical optimum then moved towards the theoretical value.
- Support Vector Machine: primal and dual, soft margin with slack, C by cross validation. Linear, polynomial and RBF kernels compared — RBF won. F1 0.784.





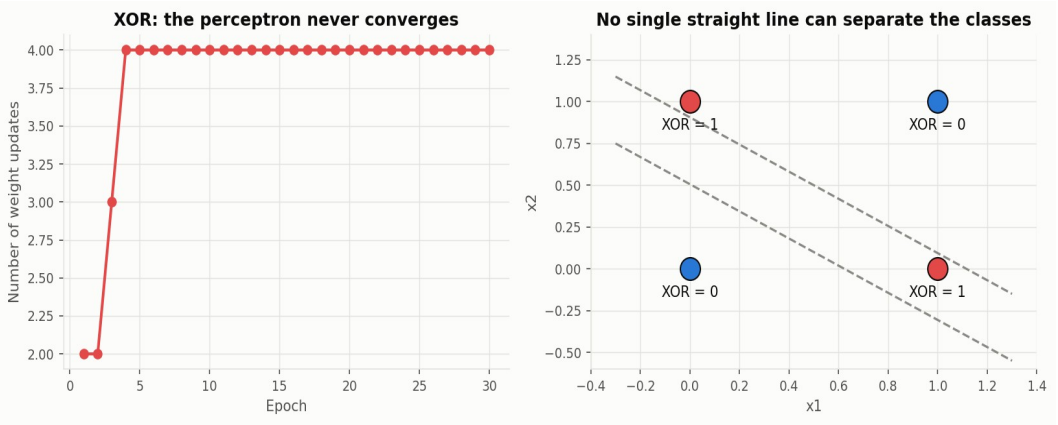
7

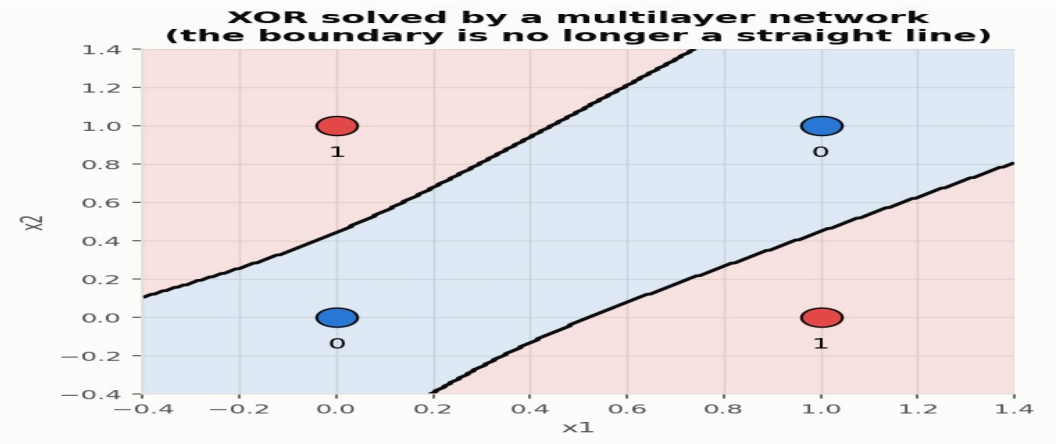
Unit IV — Neural networks

Unit IV

McCulloch–Pitts to backpropagation, and computational learning theory

- Built AND, OR and NOT from a single McCulloch–Pitts unit, then applied the perceptron learning rule — it converges on all three, as the convergence theorem guarantees.
- XOR is not linearly separable. We ran the perceptron on it and watched the weights oscillate forever. This is the historical result that stalled neural network research, reproduced on our own code rather than quoted.
- A multilayer perceptron with one hidden layer solves XOR immediately — the hidden layer bends the decision boundary.
- On the fraud data: MLP trained by backpropagation with gradient descent, loss curve monitored for convergence. F1 0.816, 104 of 142 caught with 9 false alarms.
- Computational learning theory applied: VC dimension as a measure of capacity, and a PAC bound to estimate how much data a model of that capacity needs.





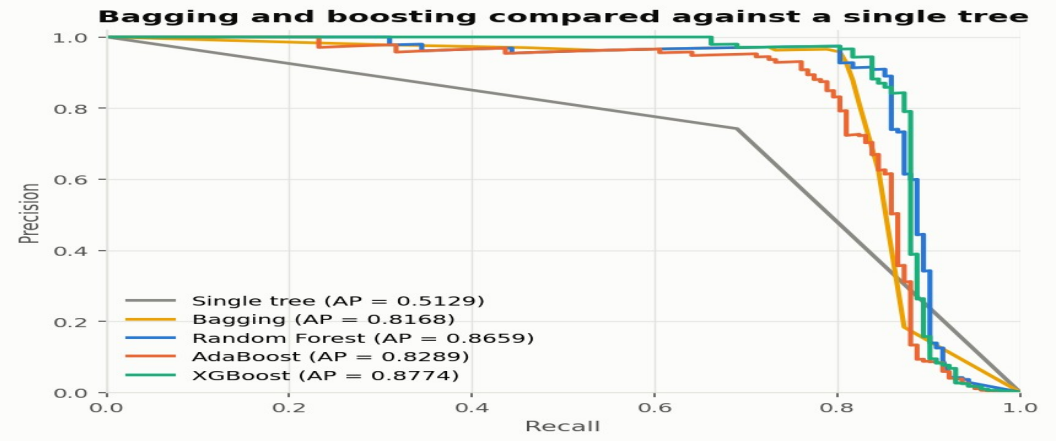
8

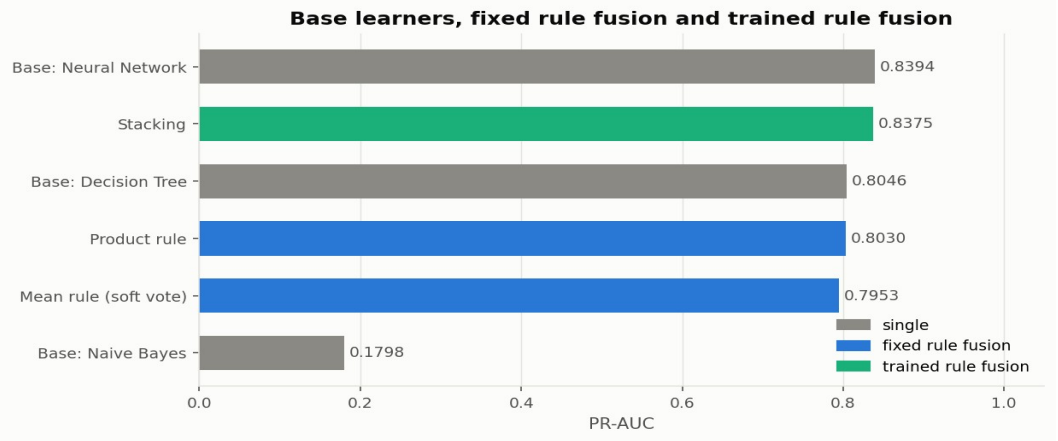
Unit V — Ensembles, rule fusion and clustering

Unit V

Where the best result came from

- Bagging — models trained independently on bootstrap samples, then averaged. Reduces variance.
- Random Forest — bagging plus a random feature restriction at each split, which decorrelates the trees. The most precise model: only 3 false alarms.
- AdaBoost and XGBoost — models trained in sequence, each correcting the last. Reduces bias. XGBoost weights the rare class with scale_pos_weight.
  - XGBoost is the best model in the project: F1 0.879, catching 116 of 142 frauds with 6 false alarms.
- Fixed-rule fusion (majority, mean, product, max, min) reached F1 0.809; trained-rule fusion (stacking) reached 0.759.
  - Neither beat XGBoost alone — a useful negative result. Fusion helps when base models make different mistakes; ours mostly made the same ones.
- K-means and hierarchical clustering found a real signal without labels (recall 0.627) but at precision 0.104 — useful for exploration, not deployment.





9

All twelve models on the same test set

Results

85,118 transactions · 142 frauds · seed 77 · ranked by F1-score

Model	Recall	Precision	F1	PR-AUC	Caught	False alarms
XGBoost	**0.817**	**0.951**	**0.879**	**0.877**	**116**	**6**
AdaBoost	0.761	0.923	0.834	0.829	108	9
Random Forest	0.725	0.972	0.831	0.866	103	3
Decision Tree	0.782	0.867	0.822	0.808	111	17
Neural Network (MLP)	0.732	0.920	0.816	0.850	104	9
Voting (fixed rule fusion)	0.746	0.883	0.809	—	106	14
k-Nearest Neighbour	0.761	0.844	0.800	0.790	108	20
Support Vector Machine	0.739	0.833	0.784	0.784	105	21
Stacking (trained rule fusion)	0.634	0.947	0.759	0.837	90	5
Perceptron	0.838	0.676	0.748	—	119	57
Naive Bayes	0.775	0.113	0.197	0.183	110	865
K-Means (unsupervised)	0.627	0.104	0.179	0.157	89	763

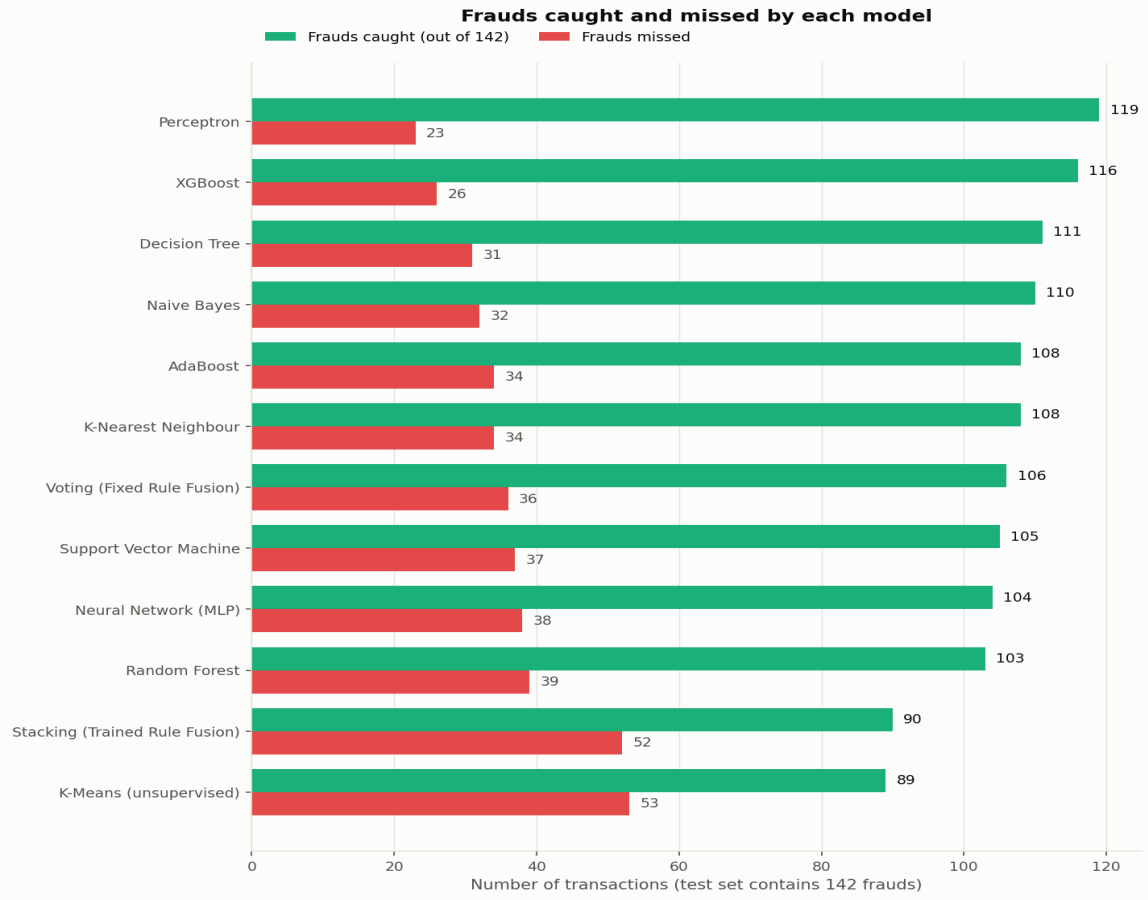
Accuracy is deliberately absent. Every model above scores over 98.9% accuracy, including the two useless ones at the bottom.

10

The same result in transactions, not ratios

Results

What each model would mean for the bank



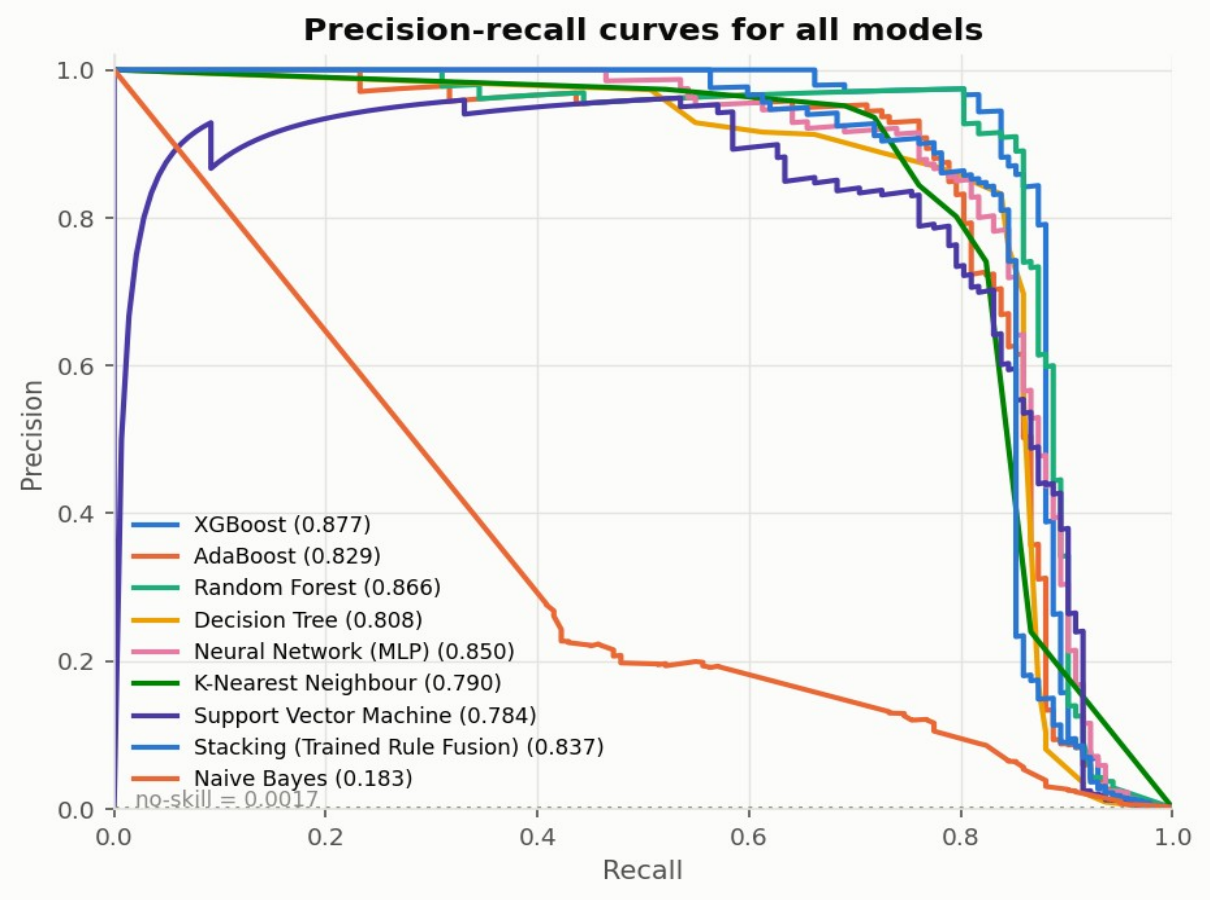
- XGBoost catches 116 of 142 frauds and raises 6 false alarms — an analyst reviews 122 transactions and 95% of them are real fraud.
- Random Forest is the most precise, with only 3 false alarms, but misses 39 frauds. It is the conservative choice.
- The Decision Tree catches more than Random Forest (111 vs 103) but raises 17 false alarms instead of 3.
- Naive Bayes catches 110 frauds and raises 865 false alarms. The recall looks fine; the workload makes it unusable.
- Which model is 'best' depends on what a false alarm costs. That is why the demo lets the threshold be moved live.

11

Precision–recall curves

The whole trade-off, not one operating point

Results



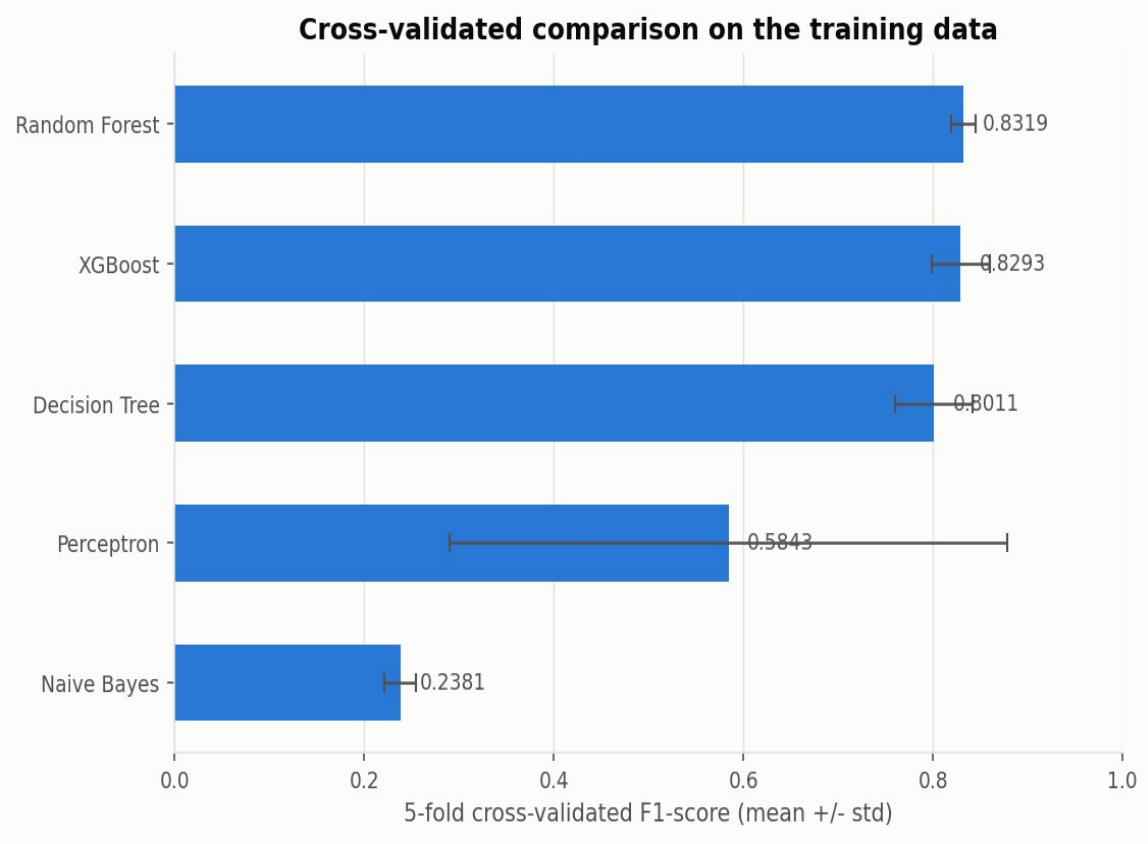
- Each curve shows what precision a model can hold at every level of recall.
- A single F1-score is one point on this curve. The curve is the honest picture.
- The baseline for a random classifier is the fraud rate itself — 0.0017. Every supervised model sits far above it.
- XGBoost dominates across almost the whole range, which is why it also wins on PR-AUC (0.877).
- Naive Bayes and K-means sit near the bottom, consistent with their precision.

12

Is the ranking real, or an accident of one split?

Validation

Five-fold stratified cross validation



Model	CV F1 mean	Std dev	Time
Random Forest	0.832	± 0.013	59 s
XGBoost	0.829	± 0.030	9 s
Decision Tree	0.801	± 0.041	22 s
Perceptron	0.584	± 0.294	2 s
Naive Bayes	0.238	± 0.017	1 s

- The top three keep their order under cross validation, so the test-set result is not a fluke.
- The Perceptron's ±0.294 standard deviation is the number to notice — wildly unstable between folds, which is what non-convergence looks like in practice.
- Random Forest edges XGBoost on mean CV F1 but takes six times longer to train.

13

Three findings worth pointing out

Findings

Course theory, confirmed on real data

01

The perceptron really does fail on XOR

We did not quote the convergence theorem — we ran the perceptron on XOR and watched it never converge, then solved it with one hidden layer. Unit IV, reproduced rather than cited.

02

The curse of dimensionality is measurable here

k-NN's performance degrades as attributes are added, on real transaction data rather than a synthetic example. Distances converge and neighbourhoods stop being informative. Unit II.

03

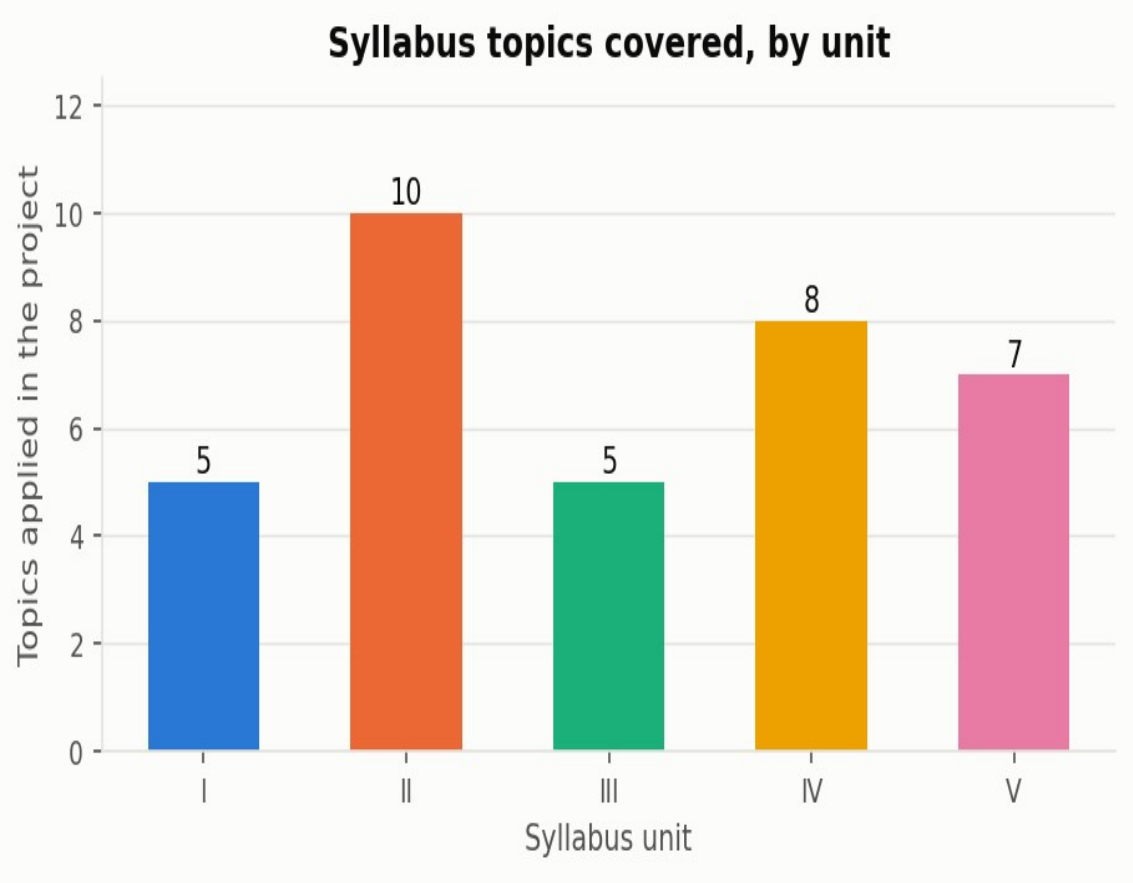
Bayes optimal decisions need calibrated probabilities

The cost-derived threshold did not work on raw naive Bayes output, because those outputs are not real probabilities. Isotonic calibration fixed it. The theory is right; its precondition is easy to miss. Unit III.

14 Coverage of the course

Syllabus

Every unit is implemented, not just mentioned



Unit	Topics implemented	Notebook
I	Training / testing / validation, CV, overfitting	01, 03, 04, 10
II	Decision tree ID3, k-NN, feature selection, PCA, LDA	02, 03, 04
III	Bayes theorem, Bayes optimal decisions, SVM	05, 06
IV	M–P neuron, perceptron, backpropagation, VC / PAC	07
V	Bagging, RF, AdaBoost, XGBoost, fusion, clustering	08, 09

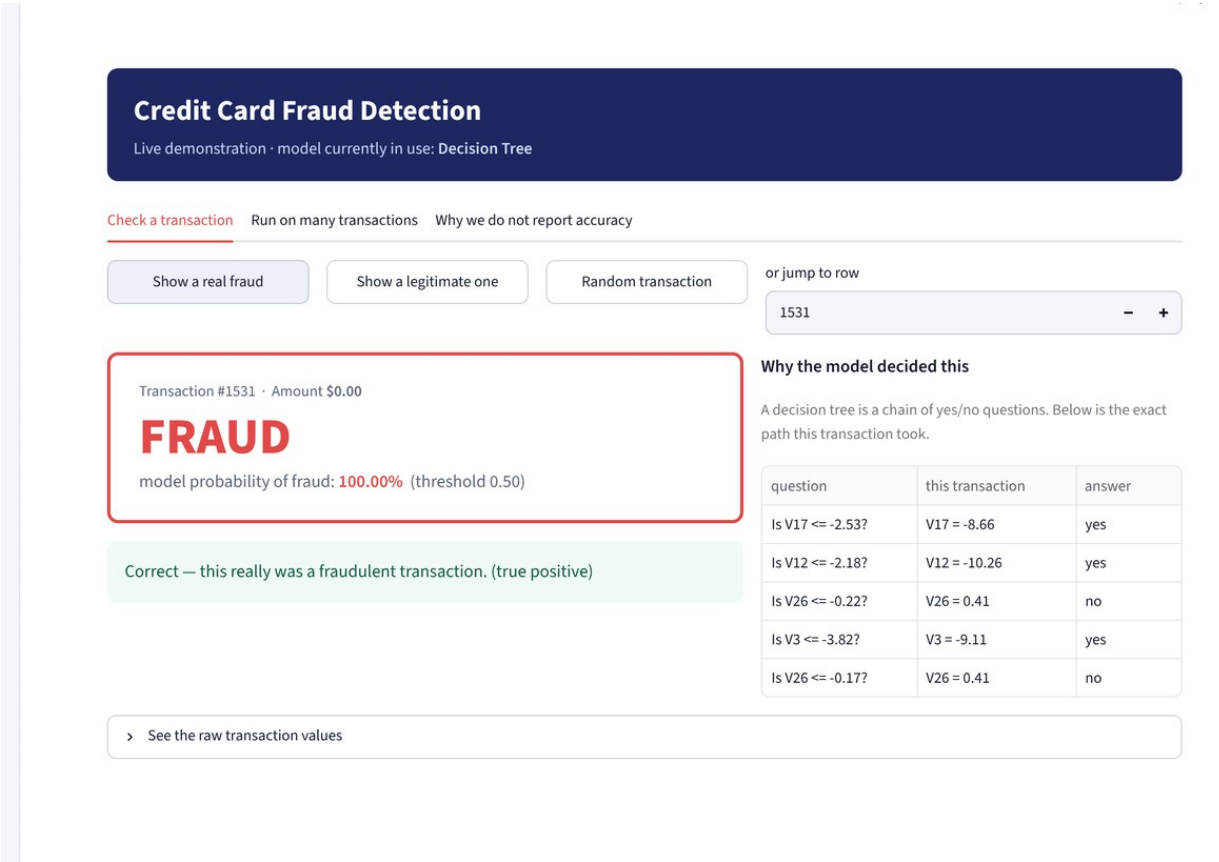
35 syllabus topics appear in the code, each traceable to the notebook that implements it. The full mapping is in Results/tables/syllabus_coverage.csv.

15 The working system

A live app, not only notebooks

Demo

- The trained models are saved to disk and loaded by a Streamlit app, so it opens instantly.
- Pick any transaction: the chosen model gives a verdict with its confidence, then reveals whether it was right.
- For the Decision Tree the app prints the exact chain of questions the tree asked — the model's reasoning, readable on screen.
- The whole sample can be scored at once into a confusion matrix, and moving the threshold slider changes it live: frauds caught rises, false alarms rise with it.
- The sample is deliberately fraud-heavy so there is something to see; the app says so, and shows the true full-test-set figures alongside.
- Run it with: streamlit run Demo/app.py



A fraud flagged, with the exact rule the decision tree followed

16

Limitations, and where this would go next

Limits

What the project does not claim

Limitations

- Features are anonymised — V1–V28 are PCA components, so we cannot name the real attribute behind a decision.
- Two days of 2013 data from one region. Fraud patterns drift; this would need retraining on current data.
- A single held-out split plus cross validation. No temporal validation, which production would require.
- The 50:1 cost ratio is a reasonable assumption, not a figure measured at a real bank.
- No deployment work: no API, no monitoring, no retraining schedule, no review queue.
- Fusion did not beat the best single model — reported as a negative result rather than a claimed advantage.

Future work

Temporal validation	Train on earlier transactions, test on later ones.
Resampling study	SMOTE, ADASYN and undersampling, measured inside the folds.
Cost-sensitive learning	Put the asymmetric cost into the training objective itself.
Calibration everywhere	So thresholds are comparable across all models.
Explainability	SHAP values, giving the ensembles what the tree already has.
Concept drift	Monitor over time and retrain when performance degrades.

Conclusion

- Twelve models from all five units of the syllabus, trained and compared on one identical, leakage-free protocol.
- XGBoost performed best: recall 0.817, precision 0.951, F1 0.879 — 116 of 142 frauds caught with 6 false alarms.
- Ensembles beat single learners; boosting beat bagging; fusion beat neither.
- Accuracy was rejected throughout, and we showed why: a classifier that detects nothing scores 99.83%.
- Three pieces of course theory were reproduced on real data rather than quoted — the XOR limitation, the curse of dimensionality, and the precondition behind Bayes optimal decisions.
- Everything is reproducible from the repository with a fixed seed of 77, and the system runs as a live app.

Thank you

Questions?

Akshita Modi

AP24110010386

Lakshay

AP24110010677

Mohana Krishna Reddy Pittu

AP24110011262

Nadimpally Dheva Harshit

AP24110011553

B.Tech Computer Science and Engineering · SRM University – AP · Supervisor: Mr. Ajay Dilip Kumar Marapatla

github.com/lakshay8toic/Credit-Card-Fraud-Detection